

BACHELOR THESIS
ARTIFICIAL INTELLIGENCE

Radboud University



Using Diffusion Models to
Reconstruct Observations in Deep
Reinforcement Learning

Author:

D. Wichmann
s1119123
daan.wichmann@ru.nl

First supervisor:

dr. L. Ambrogioni
Donders Institute for Brain,
Cognition and Behaviour
luca.ambrogioni@donders.ru.nl

Second reader:

dr. L.F.M. ten Bosch
Centre for Language Studies
louis.tenbosch@ru.nl



June 30, 2026

Abstract

The deep Q-learning algorithm has been one of the most successful breakthroughs in the field of machine learning so far, introducing the first successful application of deep neural networks in reinforcement learning. However, the use of deep neural networks is known to be a double-edged sword. While being able to map highly-dimensional sensory input to action values with high accuracy, small perturbations in the input, such as sensor noise, can heavily degrade these estimations. Additionally, most problems in reinforcement learning suffer from partial observability, meaning that the state of the environment is not fully visible, and the agent only receives partial observations, degrading performance even further. Most research tries to solve the problem of noisy and partial observations using some form of memory, e.g. by using recurrency or transformer-based networks. This thesis proposes to solve the problem of noisy and partial observation not by the use of recurrence, but by learning a belief of what has been seen before using generative modelling. I show that a diffusion model trained on the states of the environment can achieve peak signal-to-noise ratios up to 57 dB on denoising observations corrupted with Gaussian noise. Additionally, it was able to improve the obtained total rewards of a baseline DDQN agent up to 10-82 times in Atari Space Invaders and Breakout when presented with observations corrupted with low levels of Gaussian noise. Additionally, I present a novel diffusion sampler based on Heun's method, able to inpaint accurate in-game scores with a mean difference of 7.7 point, and full frames with a mean squared error of $1.3 \cdot 10^{-3}$. My implementation is available at <https://github.com/D21W12/Diffusion-Deep-Reinforcement-Learning>.

Contents

1	Introduction	2
2	Background	4
2.1	Reinforcement Learning	4
2.1.1	Deep Reinforcement Learning	4
2.1.2	Noise & Partial Observability	5
2.2	Diffusion Models	5
2.2.1	Diffusion Score Matching using ODEs	6
2.2.2	Image Inpainting using RePaint	7
3	Methods	9
3.1	Diffusion Agent	9
3.2	Deep Reinforcement Learning	10
3.3	Diffusion Models	11
3.4	Denoising Methods	12
3.5	RePaint Adaptation for EDM	12
4	Experiments	14
4.1	Denoising Noisy Observations	14
4.2	Diffusion Agent	15
4.3	Understanding of Environment Dynamics	17
4.3.1	Inferring the In-Game Score	17
4.3.2	Predicting the Future	19
5	Discussion	20
A	Additional Results	25
B	Deep Reinforcement Learning	30
C	Diffusion Models	32
D	Denoising Methods	34

Chapter 1

Introduction

The common video game enthusiast has played the popular racing game of Mario Kart at some point in their life, and has subsequently felt the wrath of the 'blooper' power-up. The blooper power-up temporarily partially obscures the player's screen with black ink, making it more difficult to play. In the field of reinforcement learning (RL) this is known as partial observability. When an environment is partially observable, the agent is only presented with noisy or partial observations, which do not include the full state information. Real-world applications of reinforcement learning often suffer from partial observability, as agents rarely have access to noiseless states or complete observations. For instance, noise inherent in physical sensors can prevent agents from observing the true state of the environment.

Partial observability has been a longstanding problem in the field of RL, as many tasks require a high level of robustness. Moreover, state-of-the-art deep reinforcement learning (DRL) algorithms are well-known to be highly sensitive to small perturbations [7, 21]. Most solutions rely on recurrency, or transformer-based models [5]. This thesis proposes a different solution to the problem of partial observability, by using an emerging class of generative models, the diffusion model.

Diffusion models [6, 15, 16, 17] have emerged as a powerful new class of generative models, achieving state-of-the-art performance on a wide range of generative tasks, such as image synthesis, video generation, and molecule design. They are able to model arbitrarily complex data distributions, by trying to reverse a forward diffusion process inspired by non equilibrium thermodynamics [15], the discipline that studies how gasses spread in air, or ink in water. After training, diffusion models are able to generate high-fidelity samples from the learned data distribution by sampling from a simple prior (often a Gaussian) and transforming the samples iteratively in a noise removal process. The task of removing noise from data points is often referred to as denoising.

Another popular application of diffusion models is image inpainting. Image inpainting is the task of filling in a masked/missing area in an image, using semantically correct details. A successful approach for image inpainting using diffusion models is RePaint, proposed by [11].

With the ability of image inpainting and denoising, one could see the potential of diffusion models to solve the problem of partial observability by reconstructing states from partial observations. While diffusion models have already found their way into the field of reinforcement learning [22], mainly as policies, planners, and data synthesizers, their potential to reconstruct states from noisy and partial observations has yet to be explored. This thesis aims to answer the following research question:

Can a diffusion model improve performance of deep reinforcement learning by removing noise from observations?

This thesis will combine DeepMind’s work in DRL [12] and NVIDIA’s research in elucidating the design space of diffusion models (EDM) [8] to construct a series of experiments to answer the posed research question. The main contributions of my work are:

1. A novel framework for using diffusion models to reconstruct observations and improve performance in deep reinforcement learning.
2. Two methods for denoising images with Gaussian noise using the EDM formulation of diffusion.
3. An adapted sampler implementing RePaint for free-form image inpainting using the EDM formulation of diffusion.

In addition to answering the posed research question, this thesis will also show that the same diffusion models trained on states of the environment are successful in inpainting realistic in-game scores and predicting frames that obey system dynamics, showing they have learned a strong generative prior.

In the chapters that follow, first, a more detailed description regarding the preliminaries of DRL and diffusion models will be given. Chapter three will propose the solution framework, denoising methods, and adapted sampler for inpainting, followed by a chapter setting up four different experiments and displaying their results. In the last chapter, the results and limitations of the experiments are discussed, and recommendations for future research are provided.

Chapter 2

Background

2.1 Reinforcement Learning

Reinforcement learning [18] is an important branch of machine learning, where agents try to behave optimally in sequential decision problems by directly interacting with the environment. At each timestep t , agents are presented with a state s_t from the state space $\mathcal{S}$. Based on this state agents try to select an action a_t from the action space $\mathcal{A}$, which transitions them to state s_{t+1} . After taking an action, the agent receives a reward r_t according to a reward function R , designed to reward desired behavior (positive rewards), and punish undesired behavior (negative rewards). We can describe how good a certain state-action pair is using the notion of a Q-function, which defines the expected reward given a state-action pair and following some mapping from states to actions π (a policy) as $Q(s, a) = \mathbb{E}[R_t | s_t = s, a_t = a, \pi]$, i.e. how rewarding is it to take action a in state s if we choose our subsequent actions based on the mapping π ? The learning goal of RL is often formulated as learning the optimal Q-value function $Q^*(s, a)$, which is the optimal value achievable following some policy π . The policy an agent has to follow is then simply a greedy strategy, which takes the action that maximizes $Q^*(s, a)$.

2.1.1 Deep Reinforcement Learning

While estimating the optimal Q-functions for smaller finite state spaces is plausible, this problem becomes substantially harder when the state space is continuous, like image data. In recent years, a new paradigm of RL, deep reinforcement learning, has achieved human-level control at a variety of complex control tasks. The success of DRL was sparked by Mnih et al. [12], who introduced deep Q-learning. Deep Q-learning estimates Q-values using a deep neural network (NN), which they refer to as the deep Q-network (DQN). The DQN receives the current state s_t as input, and outputs a Q-value for every possible action $a \in \mathcal{A}$. Because of the use of NNs, the states can range from simple discrete feature vectors, to complex continuous image data.

The success of the DQN algorithm can be partially credited to two novel contributions, a biologically inspired experience replay, and the target network. Experience

replay stores the last N transitions $e(s_t, a_t, r_t, s_{t+1})$ in memory. These are then sampled during training to break any correlations present in sequential interactions with the environment. Secondly, they introduce a second network, the target network. The target network periodically copies the weights of the DQN, and is used to break correlations in the loss function.

DQN has been shown to outperform professional humans at a series of Atari 2600 games [12] from the Arcade Learning Environment (ALE) [2]. The ALE has stayed a relevant benchmarking environment for RL research up until this day, alongside DQN as a popular baseline.

Follow-up research by Hasselt et al. [19] further improved the performance of the DQN algorithm by providing the double DQN (DDQN) loss function, inspired by double Q-learning [4]. The loss function can be found in appendix B.2.

2.1.2 Noise & Partial Observability

While the use of NNs is one of the biggest strengths of DRL, it is also its biggest weakness. NNs are known to be highly sensitive to small perturbations of the input [7, 21]. This decreases the accuracy of the approximated Q-values, degrading performance of the agent. In addition, while many RL agents can often be trained in simulations with perfect observations, in real-world applications, perfect observations can rarely be assumed. For example, think of a leaf blocking a camera sensor of a self-driving car, or faulty hardware introducing unwanted noise.

When the full state information of the environment is not available to the agent, the environment becomes partially observable. In partially observable environments the agent receives an observation $o \sim \mathcal{O}$ instead of the state $s \sim \mathcal{S}$, containing only incomplete information about the current state of the environment.

In the case of the leaf blocking the camera of the self-driving car, we would not want the car to immediately give up on making well-informed decisions. Most solutions solve the problem of partial observability by giving the agent some form of access to information from the past, e.g., by using recurrency [5].

Instead of using recurrency to allow the agent to know what was there before the observations got obscured, it could be beneficial to let the agent reason about what could be there given its prior beliefs about the environment. This research tries to solve the problem of partial and noisy observations, not by using a form of memory but, by allowing the agent to project noisy or partial observations on to a learned distribution, using generative modelling.

2.2 Diffusion Models

The common objective of generative modelling involves modelling highly complex datasets using a family of data distributions. The purpose of creating such models

is often to be able to sample novel high-fidelity data. While some data distributions, e.g. the height of students, may easily be modeled using a mix of Gaussians, others, like the distribution of all natural images, are substantially harder.

Assume the goal of sampling a novel data point from a complex target distribution $p_{data}(\mathbf{x})$ and the existence of a function $F(\mathbf{x})$ which estimates the gradient of this distribution $\nabla_{\mathbf{x}} p_{data}(\mathbf{x})$. We could then start by sampling pure noise from a Gaussian $\mathbf{x} \sim \mathcal{N}(0, I)$ and move $\mathbf{x}$ progressively to positions of higher probability with $F(\mathbf{x})$ using gradient ascent. The main problem is that many complex data distributions, like image data, are highly concentrated around steep modes. This results in uninformative gradient estimations for low probability areas, as the majority of the probability landscape is flat.

To solve this problem, diffusion models diffuse the probability density to a simple prior distribution with varying levels of noise σ to obtain a set of diffused marginal distributions $\{p(\mathbf{x}; \sigma(t_i))\}_{i=0}^N$ in a process called the forward diffusion process. $\sigma(t_i)$ is a function representing the noise of the distribution at the i th timestep t_i . At $t_{i=0}$ the distribution approximately follows the target distribution $p(\mathbf{x}; \sigma(t_0)) \approx p_{data}(\mathbf{x})$, and at $t_{i=N}$ the simple prior distribution. When the peaks of the target distribution are diffused to cover more of the probability landscape, $F(\mathbf{x})$ becomes more informative. However, following the gradient of the diffused distribution $p(\mathbf{x}; \sigma(t_N))$, the sample may never arrive at one of the true peaks of the original target distribution. That is why during inference, the fully diffused distribution is diffused backwards in time to obtain the original target distribution. The process of diffusing the distribution backwards in time is called the backward process. In order to account for the marginal distributions, the gradient estimation is conditioned on σ and becomes $F(\mathbf{x}, \sigma)$.

Many different formulations for diffusion models exist, of which the three most dominant are denoising diffusion probabilistic models (DDPM) [6], score-based generative models (SGM) [16], and stochastic differential equations (Score SDE) [8, 17]. This thesis uses the EDM formulation of diffusion, which is a special case of the latter.

2.2.1 Diffusion Score Matching using ODEs

Song et al. [17] showed that how data points move through time in both the forward and backward diffusion process can be described using a stochastic differential equation (SDE). This allows the sampling process to be implemented using numerical integration. An equivalent deterministic version of the backward SDE exists as an ordinary differential equation, namely the probability flow ODE.

Karras et al. [8] proposed a novel design space which identifies all concrete design components of diffusion models, including a reparameterization of the probability flow ODE. Alongside this design space, their work proposes a set of highly performing choices for these components. Their formulation is named the EDM formulation of diffusion. This thesis will adopt the EDM formulation of diffusion, as it is highly

optimized.

Reparameterization of the Probability Flow ODE

Karras et al. [8] showed that the probability flow ODE can be reparameterized as the following ODE.

$$d\mathbf{x} = [\frac{\dot{s}(t)}{s(t)} - s(t)^2 \dot{\sigma}(t) \sigma(t) \nabla_{\mathbf{x}} \log p(\mathbf{x}; \sigma(t))] dt \quad (2.1)$$

In this equation the noise schedule $\sigma(t)$ controls the noise of the distribution during the diffusion process, $s(t)$ is the scaling function, scaling input over time, and $p(\mathbf{x}; \sigma(t))$ represent the marginal distribution at time t .

The most important part of this equation is called the score function (or score) $\nabla_{\mathbf{x}} \log p(\mathbf{x}; \sigma(t))$, the log gradient over the marginal distribution $p(\mathbf{x}; \sigma(t))$. The score tells us how to move from areas of low probability density, to areas of higher probability density. The score can be estimated using a NN via score matching [16].

Sampling using the Probability Flow ODE

Using the score, the probability flow ODE can be discretised to sample from the learned distribution using any off-the-shelf numerical integrator. In the context of image sampling, Karras et al. [8] found that using Heun’s 2nd order method [1] gives the best trade-off between the number of function evaluations (NFE) of the score and the accumulated error. In order to solve the ODE with numerical integration, a discretisation of the ODE needs to be specified. Karras et al. propose a handcrafted discretisation running the ODE in N steps from noise level σ_{max} to σ_{min} . The minimal and maximum noise levels, as well as number of steps, are all configurable hyperparameters of the discretisation. Both the algorithm for sampling using Heun’s method and the definition of the discretisation can be found in appendix C.1.

Denoising Network and Training

In order to approximate the score from training data, diffusion models make use of NNs, named score networks. It can be shown that predicting the score is related to predicting a clean image from a noisier version of itself (Equation C.1) [8] using a denoising network $D_{\theta}(\mathbf{x}, \sigma)$ (including optional pre- and post-processing of the image). The loss function can be found in Appendix C.2.

Most denoising networks follow the popular U-net [14] backbone architecture combined with self-attention [20]. Later work of Karras et al. [9] proposed a redesign of the denoising network architecture for EDM, improving the training dynamics of diffusion for image data even further.

2.2.2 Image Inpainting using RePaint

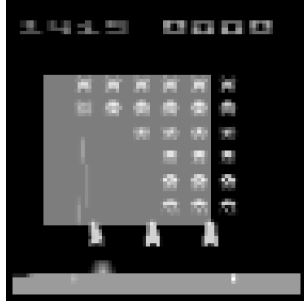
While Diffusion Models are primarily being used for image sampling, they are also fit in a range of other generative applications. One of these is free-form image

inpainting, which is the task of adding new content to an image in the regions specified by an arbitrary binary mask. Lugmayr et al. [11] proposed RePaint, a DDPM-based [6] inpainting approach applicable to any pretrained diffusion model. It alters the DDPM sampling algorithm by conditioning the unknown regions on the known. Using a binary mask m , at each timestep i of the sampling process x_i is set according to the following definition.

$$x_i = m \odot x_i^{known} + (1 - m) \odot x_i^{unknown} \quad (2.2)$$

where $x_t^{unknown}$ is the image sampled using the denoising network and x_t^{known} is directly computed using the input image and analytical solution of the forward diffusion process.

In order to harmonize the content of the masked regions with the content of the unmasked region, every time step the sample is resampled U times by diffusing the sample forward in time. An adaptation of RePaint for the EDM formulation will be proposed in section 3.5, allowing evaluation on the diffusion model’s ability to inpaint missing details from obscured observations.



(a) The original, overlay-ed, image.



(b) The inpainted image.

Figure 2.1: This figure demonstrates an example of free-form image inpainting. Subfigure 2.1a is an image from the Atari 2600 game Space Invaders. The mask used for inpainting is overlay-ed in opaque grey. Subfigure 2.1b shows the inpainted image, inpainted using the methods proposed in Section 3.5 inspired by Lugmyar et al. [11]. It can be seen that the masked region is inpainted with different, however still plausible, details in contrast to the original image.

Chapter 3

Methods

3.1 Diffusion Agent

The intuition of using diffusion models in DRL is to equip agents with a prior belief over the possible states of the environment. By training a diffusion model on states from the environment, the agent can use this model during inference to project imperfect observations onto the learned distribution (e.g. by image inpainting or denoising). Additionally, if this distribution is learned using states used during training of the original agent, the reconstructed states are hypothesized to be closer to what the agent has encountered before, allowing for more accurate Q-value estimations. The replay memory of the DQN algorithm provides a natural candidate as a training dataset of the diffusion model, as it is a readily available buffer of states used during training of the agent.

To put the solution in a more formal setting, the goal of training the diffusion model is to learn a distribution $p(s)$ over the states of the environment $s \sim \mathcal{S}$. The model can then be utilized in applications where the environment returns noisy or partial observations o from which the model tries to predict the underlying clean states s , using a diffusion-based reconstruction method.

This thesis refers to the resulting agent as a diffusion agent, or diffusion DRL. A schematic overview of the interaction of a diffusion agent with the environment can be found in Figure 3.1.

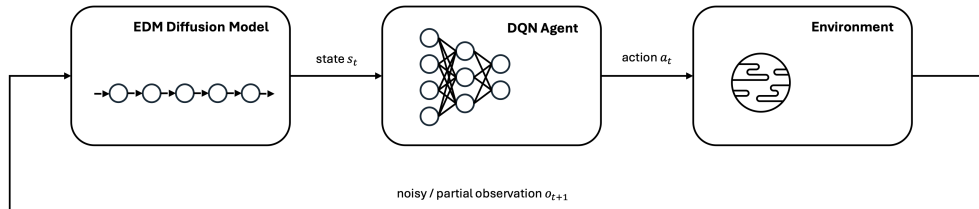


Figure 3.1: Schematic overview of the application of a diffusion agent in a partially observable environment. Solid lines represent agent control.

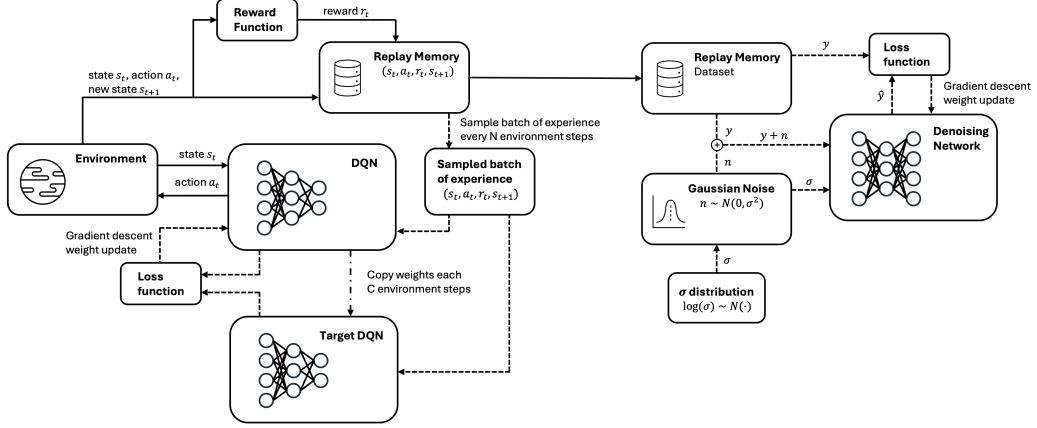


Figure 3.2: Schematic overview of the training process of a diffusion agent. Solid lines represent agent control, and dashed lines indicate information flow during training. Figure adapted from Brandi et al. [3].

The choices of RL algorithm, diffusion formulations, and reconstruction methods are design choices which can be made based on application. This thesis will use the DQN algorithm [12] and the EDM formulation of diffusion [8] to investigate the performance of the proposed solution on two Atari 2600 games (Breakout and Space Invaders). A schematic overview of the training procedure of diffusion RL, as used in this study, can be found in Figure 3.2. More details about the implementation of these methods follow in the succeeding two sections. For the reconstruction methods used, this thesis proposes two denoising methods for denoising noisy observations in Section 3.4, and an adaptation of RePaint to inpaint partial observations in Section 3.5.

3.2 Deep Reinforcement Learning

The RL agents used in the experiments are implemented and trained using the DQN algorithm and DDQN loss, following the methods as proposed by Mnih et al. [12] and Hasselt et al. [19], respectively. The agents are trained on two emulated Atari 2600 games (Breakout and Space Invaders; one for each of the two games) from the Arcade Learning Environment [2]. The specific architecture of the DQN can be found in Appendix B.

Environment. At each environment step t , the agent receives the state of the environment as an image consisting of the 4 most recent 84×84 grayscale frames from the emulator stacked on top of each other $s_t \in \mathbb{R}^{4 \times 84 \times 84}$. Based on this state, the agent sequentially selects an action a_t from a set of discrete controls provided by the emulator using an ϵ -greedy policy, picking a random action with probability ϵ and following a greedy policy with probability $1 - \epsilon$.

Training. The agents learn by minimizing the DDQN loss (Equation B.1) [19] on batches of experience sampled from the replay memory, using gradient descent.

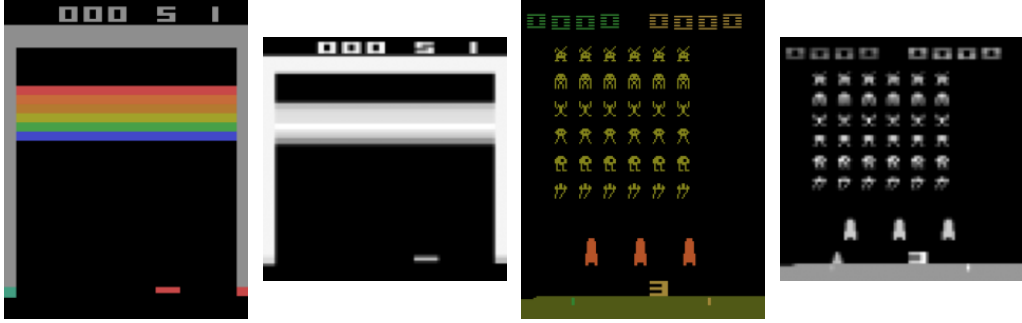


Figure 3.3: Two screenshots of the games Breakout and Space Invaders, respectively, with their associated states as observed by the DQN.

This thesis makes two minimal changes to the training methods proposed by Mnih et al. [12]. The originally used RMSProp optimizer is replaced by AdamW, in combination with a lowered learning rate (LR) of $2.5 \cdot 10^{-5}$. The agents are trained for a total of 20 million environment steps. All training, preprocessing, and hyperparameters values can be found in Appendix B.

3.3 Diffusion Models

For each agent in the experiment, one diffusion model is trained on a subset of states in the replay memory. The diffusion models are implemented and trained using the EDM formulation of diffusion as proposed by Karras et al. [8, 9].

Training. Each diffusion model is trained on a subset of 500 thousand states (i.e. images) drawn from the RL agent’s replay memory. It minimizes the loss described in Equation C.2b using the Adam optimizer. To exploit GPU efficiency for tensor operations, the images are padded to $\mathbb{R}^{4 \times 88 \times 88}$. The LR is set to 10^{-3} following the CIFAR-10 configuration in Karras et al. No LR ramp-up or EMA half-life is used. All other training hyperparameters are set as originally proposed. As the diffusion models trained in this experiment are not trained using any label conditioning, they are unconditional. A more detailed description of hyperparameter settings and training details can be found in Appendix C.3.

Denoising Network. The architecture of the denoising network follows the network design as proposed in follow-up research by Karras et al. [9]. The specific architecture configuration can be found in Appendix C.3.

3.4 Denoising Methods

In order to predict noiseless states from noisy observations in the experiments, the models need some method of removing Gaussian noise that does not require a conditional training or finetuning. In the following subsections I propose two denoising methods for diffusion models formulated with EDM.

Full Denoising. Following Karras et al. [8], during inference (i.e. sampling new images) the model starts from pure noise $x_0 \sim \mathcal{N}(\mathbf{0}, \sigma_{max}^2 s^2(t_0)\mathbf{I})$, and iteratively transforms it to images in N steps using the denoising network and Heun’s method. The first denoising method, which I name full denoising, makes use of the same sampler, but instead starts from the noisy observation o_{noisy} and projects it onto the learned distribution by transforming it from the level of noise present in the observation σ_{noise} . In other words, we change the Heun sampler to set the starting point to o_{noisy} and the starting noise level in the schedule to $\sigma_{max} = \sigma_{noise}$.

Karras et al. proposes the number of sampler steps to be set to $N = 32$, sitting perfectly on the trade-of between NFE and sample quality. However, denoising only partially noisy images should theoretically require less work, as the noise levels are drastically lower than the $\sigma_{max} = 80$ originally proposed for sampling. I find that denoising quality starts to plateau around $N \geq 8$. In the experiments N is therefore set to 8, as processing speed is highly desirable in online RL applications.

Naive Denoising. Using 8 sampler steps, however, still requires 17 expensive NN function evaluations. Therefore, I propose a second denoising method, named naive denoising, which uses only a single NFE. The denoising network $D_\theta(\mathbf{x}; \sigma)$ is trained to predict clean images directly from a noisy version of itself, corrupted exactly with the type of noise we wish to remove: Gaussian noise (Equation C.2b). We can therefore make use of this fact, by simply conditioning the network on the level of noise present in the observation, and remove the noise in a single function evaluation. Both denoising algorithms can be found in Appendix D.

3.5 RePaint Adaptation for EDM

Besides removing Gaussian noise from noisy observations, this thesis will also investigate whether the trained diffusion models have learned some degree of environment dynamics and are able to inpaint partial observations, through two inpainting tasks. This thesis adopts the RePaint inpainting approach as proposed by Lugmayr et al. [11]. As RePaint is originally proposed for DDPM-based [6] models, it needs to be adapted to be compatible with the EDM formulation using the probability flow ODE. In this section, I propose a novel sampler that implements RePaint based on Heun’s method and Karras et al.’s [8] ”churn” mechanism for stochastic image sampling.

Heun’s 2nd Order Method with Conditioning and Resampling. RePaint makes two important contributions to the sampling algorithm of DDPM: conditioning and resampling. These contributions allow it to inpaint masked images using any pretrained unconditional diffusion model. The following adaptations implement

these two contributions with Heun’s method.

Conditioning. In order to condition the masked area on the rest of the image using Heun’s method, x_i is set according to the following set of Equations in every step i of the sampling process.

$$\mathbf{x}_{i+1}^{known} \sim \mathcal{N}(\mathbf{y}, \sigma^2(t)\mathbf{I}) \quad (3.1a)$$

$$\mathbf{x}_{i+1}^{unknown} = \mathbf{Heun}(\mathbf{x}_i, i) \quad (3.1b)$$

$$\mathbf{x}_{i+1} = m \odot \mathbf{x}_{i+1}^{known} + (1 - m) \odot \mathbf{x}_{i+1}^{unknown} \quad (3.1c)$$

In these Equations $\sigma(t)$ is the noise function, y is the original image, m is a binary mask, and **Heun** is a function returning one integration step using Heun’s method (Algorithm 3). Equation 3.1c is also used within the Heun step to set $\mathbf{x}_{i+1}$ before the second order integration.

Resampling. To implement resampling, the sampler takes inspiration from Karras et al.’s stochastic sampler using churn [8]. Each sampling step i is resampled U times by adding a small amount of noise, bringing the sample back to the previous point in the discretisation $i - 1$.

$$\hat{\sigma} = \sqrt{\sigma^2(t_{i-1}) - \sigma^2(t)} \quad (3.2a)$$

$$\mathbf{x}_{i-1} \sim \mathcal{N}(\mathbf{x}_i, \hat{\sigma}^2\mathbf{I}) \quad (3.2b)$$

The two adaptations result in the sampling algorithm in Algorithm 1.

Algorithm 1 Inpainting based on Heun’s 2nd order method with an arbitrary binary mask m .

```

1: procedure REPAINTHEUNSAMPLER( $\mathbf{y}, \mathbf{m}, D_\theta(\mathbf{x}; \sigma), \sigma(t), s(t), t_{i \in \{0, \dots, N\}}\}$ )
2:    $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \sigma^2(t_0)s^2(t_0)\mathbf{I})$ 
3:   for  $i \in \{0, \dots, N - 1\}$  do
4:     for  $u \in \{0, \dots, U - 1\}$  do ▷ Apply  $U$  resamples over all time steps
5:        $\mathbf{x}_{i+1}^{known} \sim \mathcal{N}(\mathbf{y}, \sigma^2(t_{i+1})\mathbf{I})$ 
6:        $\mathbf{d}_i \leftarrow (\frac{\dot{\sigma}(t_i)}{\sigma(t_i)} + \frac{\dot{s}(t_i)}{s(t_i)})\mathbf{x}_i - \frac{\dot{\sigma}(t_i)s(t_i)}{\sigma(t_i)}D_\theta(\frac{\mathbf{x}_i}{s(t_i)}; \sigma(t_i))$ 
7:        $\mathbf{x}_{i+1}^{unknown} \leftarrow \mathbf{x}_i + (t_{i+1} - t_i)\mathbf{d}_i$ 
8:        $\mathbf{x}_{i+1} \leftarrow m \odot \mathbf{x}_{i+1}^{known} + (1 - m) \odot \mathbf{x}_{i+1}^{unknown}$ 
9:       if  $\sigma(t_{i+1}) \neq 0$  then ▷ 2nd order integration step
10:         $\mathbf{d}_i \leftarrow (\frac{\dot{\sigma}(t_{i+1})}{\sigma(t_{i+1})} + \frac{\dot{s}(t_{i+1})}{s(t_{i+1})})\mathbf{x}_{i+1} - \frac{\dot{\sigma}(t_{i+1})s(t_{i+1})}{\sigma(t_{i+1})}D_\theta(\frac{\mathbf{x}_{i+1}}{s(t_{i+1})}; \sigma(t_{i+1}))$ 
11:         $\mathbf{x}_{i+1}^{unknown} \leftarrow \mathbf{x}_i + (t_{i+1} - t_i)(\frac{1}{2}\mathbf{d}_i + \frac{1}{2}\mathbf{d}'_i)$ 
12:         $\mathbf{x}_{i+1} \leftarrow m \odot \mathbf{x}_{i+1}^{known} + (1 - m) \odot \mathbf{x}_{i+1}^{unknown}$ 
13:       if  $u < (U - 1) \wedge i < (N - 1)$  then ▷ Avoid the last resample
14:          $\hat{\sigma} \leftarrow \sqrt{\sigma^2(t_{i-1}) - \sigma^2(t)}$ 
15:          $\mathbf{x}_{i-1} \sim \mathcal{N}(\mathbf{x}_i, \hat{\sigma}^2\mathbf{I})$ 

return  $\mathbf{x}_N$ 

```

Chapter 4

Experiments

Thus far, I have proposed a solution for RL agents trained using the DQN algorithm [12, 19] to denoise and inpaint observations using an unconditional diffusion model trained with the EDM formulation of diffusion [8, 9]. In this chapter I present four different experiments evaluating and showcasing the performance of both the resulting agents and trained models.

4.1 Denoising Noisy Observations

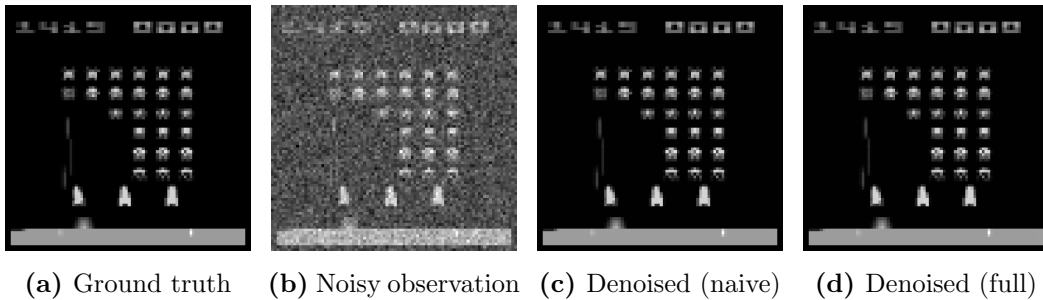


Figure 4.1: A denoising example on an observation from Space Invaders. Subfigure 4.1a shows the original ground truth observation, 4.1b the observation corrupted with Gaussian noise ($\sigma_{noise} = 0.25$), subfigure 4.1c the noisy observation denoised using naive denoising, and subfigure 4.1d using full denoising.

The first experiment aims at examining the denoising performance of the proposed denoising methods as a function of noise level σ_{noise} . For observations of each game, the peak signal-to-noise ratio (PSNR) between the noiseless observation and the denoised observation is evaluated on 100 samples for different levels of additive Gaussian noise ($\sigma_{noise} = \{0.1, 0.2, \dots, 1\}$). I compare the methods against a simple median filter baseline, which is a popular non-linear filter that aims to remove noise by replacing pixel values by the median of its neighbors in a k -sized window. Throughout all experiments in this thesis k is set to 3.

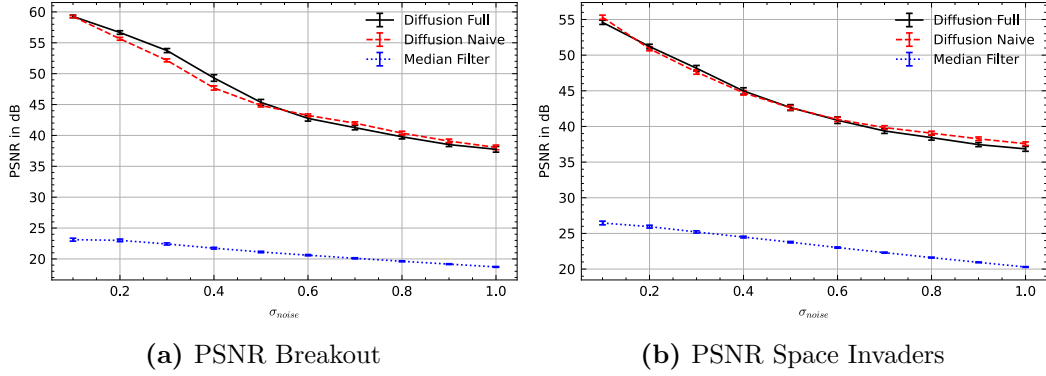


Figure 4.2: The peak signal-to-noise ratio (PSNR; in dB) of denoising observations corrupted with Gaussian noise from Breakout and Space Invaders, respectively.

Figure 4.2 shows that, for both Breakout and Space Invaders, the proposed diffusion-based denoising methods achieve high PSNRs (60-40 dB) for low levels of noise (0.1-0.4), which is a substantial improvement over the median filter baseline. Even for higher levels (0.5-1.0) of noise the diffusion-based denoising methods perform twice as well. The full and naive denoising methods show similar performance, without any significant differences. These results suggest that diffusion-based denoising performs very well in denoising noisy observations. The faster decrease in PSNR at higher noise levels for Space Invaders might be due to the higher detailing in observations compared to Breakout. The results for a subset of noise levels are summarized in table 4.1, and the full table can be found in appendix A.

Table 4.1: The peak signal-to-noise ratios (PSNR) of the denoised observations from Breakout and Space Invaders using different denoising methods.

Game	Method	$\sigma_{noise} = 0.2$	$\sigma_{noise} = 0.4$	$\sigma_{noise} = 0.6$	$\sigma_{noise} = 0.8$
Breakout	Median	23.0 $\pm$ 0.21	21.7 $\pm$ 0.14	20.6 $\pm$ 0.10	19.6 $\pm$ 0.08
	Diff (Naive)	55.6 $\pm$ 0.22	47.7 $\pm$ 0.34	43.2 $\pm$ 0.24	40.4 $\pm$ 0.30
	Diff (Full)	56.7 $\pm$ 0.29	49.3 $\pm$ 0.53	42.8 $\pm$ 0.43	39.8 $\pm$ 0.35
Space Invaders	Median	25.9 $\pm$ 0.20	24.5 $\pm$ 0.12	23.0 $\pm$ 0.09	21.6 $\pm$ 0.06
	Diff (Naive)	50.9 $\pm$ 0.29	44.7 $\pm$ 0.31	41.0 $\pm$ 0.30	39.1 $\pm$ 0.27
	Diff (Full)	51.2 $\pm$ 0.32	45.0 $\pm$ 0.45	40.9 $\pm$ 0.45	38.4 $\pm$ 0.35

4.2 Diffusion Agent

In the second experiment, I aim to answer the research question posed in the introduction of this thesis: can a diffusion model increase the performance of deep reinforcement learning? The experiment compares different agents in noisy versions of the Atari environments, adding independent Gaussian noise with σ_{noise} to the observations before they are presented. I closely follow the evaluation metrics proposed by Mnih et al. [12]. For each noise level ($\sigma_{noise} = \{0.1, 0.2, \dots, 1\}$) the agents

play 30 episodes, after which the mean total reward and mean episode length are reported. Higher levels of total reward and episode length indicate better agent performance. The following agents are compared against each other.

DDQN. The first (baseline) agent is comprised of the agent trained in section 3.2 using the DQN algorithm [12] and DDQN loss [19]. It uses an ϵ -greedy policy, picking a random action with probability $\epsilon = 0.05$, and the action that maximizes the Q-value with probability $1 - \epsilon = 0.95$.

DDQN + Median filter. The second baseline agent is comprised of the same base DDQN agent, but uses a median filter with $k = 3$ in an attempt to denoise observations before they are passed to the DQN.

Diffusion (Naive) The naive diffusion agent uses the framework proposed in section 3.3 to denoise observations with the naive denoising method using its prior belief about the environment, i.e. the model trained in 3.3. Denoised observations are then passed to the DQN, which shares its parameters with the other agents.

Diffusion (Full) The full diffusion agent differs only from the naive agent by its denoising method, using the proposed full denoising method. The amount of sampler steps for the agent N is set to 8 in this experiment.

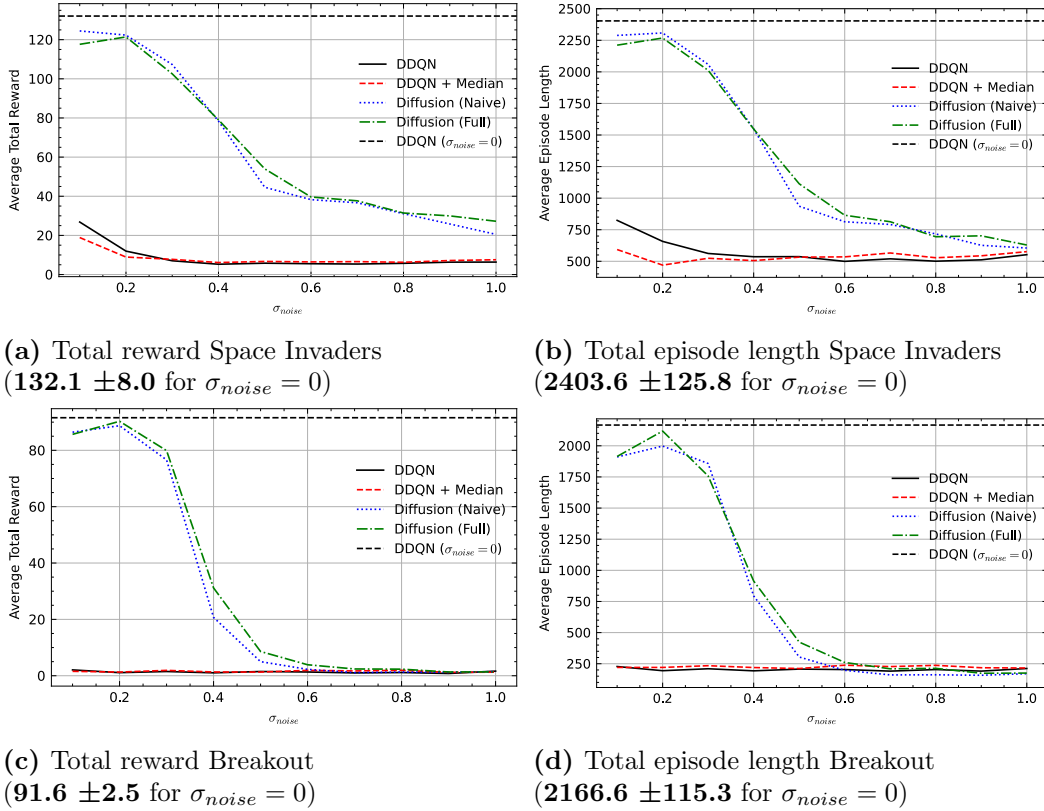


Figure 4.3

From figure 4.3 it can be seen that, for lower levels of noise (0.1-0.2), both diffusion agents facilitate performance to an almost identical level of that obtained by DDQN in noiseless environments (indicated by the dashed black line). In contrast, both the obtained total reward and episode length of both baseline agents almost completely degrade immediately. The performance of the diffusion agent on Space Invaders decreases gradually with the level of noise in the observations, while the performance on Breakout rapidly declines for noise levels $\sigma_{noise} > 0.3$. This difference in decline is likely caused by the models inability to reconstruct important details in Breakout, which heavily influence the agent’s ability to play (i.e. the ball in the game), in contrast to Space Invaders, where such important details do not exist. While in section 4.1 it can be seen that the diffusion model’s denoising performance is still relatively high for higher levels of noise ($\sigma_{noise} \geq 0.5$), the agent’s inability to perform well in environments with these levels of noise could potentially be explained by the fact that the models might hallucinate important details in these conditions. The results of the total reward are summarized in table 4.2 for a subset of noise levels. The complete tables can be found in appendix A.

Table 4.2: The mean (SE) total reward over 30 episodes for four different agents for different levels of noise in the observations σ_{noise} of Breakout and Space Invaders.

Game	Agent	$\sigma_{noise} = 0.2$	$\sigma_{noise} = 0.4$	$\sigma_{noise} = 0.6$	$\sigma_{noise} = 0.8$
Breakout	DDQN	1.1 $\pm$ 0.3	1.0 $\pm$ 0.4	1.3 $\pm$ 0.3	1.1 $\pm$ 0.4
	Median	1.3 $\pm$ 0.3	1.3 $\pm$ 0.4	1.9 $\pm$ 0.5	1.9 $\pm$ 0.5
	Diff (Naive)	88.7 $\pm$ 3.0	20.7 $\pm$ 1.2	2.3 $\pm$ 0.3	1.3 $\pm$ 0.1
	Diff (Full)	90.3 $\pm$ 2.7	31.2 $\pm$ 3.6	3.9 $\pm$ 0.4	2.3 $\pm$ 0.4
Space	DDQN	12.0 $\pm$ 1.1	5.3 $\pm$ 0.5	5.5 $\pm$ 0.5	5.7 $\pm$ 0.4
Invaders	Median	8.9 $\pm$ 0.9	6.1 $\pm$ 0.5	6.5 $\pm$ 0.5	6.3 $\pm$ 0.5
	Diff (Naive)	122.3 $\pm$ 8.1	78.4 $\pm$ 8.6	38.3 $\pm$ 3.6	31.1 $\pm$ 2.1
	Diff (Full)	121.4 $\pm$ 7.7	79.0 $\pm$ 7.3	39.6 $\pm$ 3.1	31.4 $\pm$ 2.3

4.3 Understanding of Environment Dynamics

The last two experiments aim at evaluating the degree to which the models have learned about the underlying dynamics of the environment, i.e. have they learned a reliable prior on the environment states. Unfortunately assessing the former directly is complicated, as formulating a direct metric is difficult. Therefore, the next subsections propose two inpainting tasks that evaluate the model’s ability to inpaint plausible details in masked observations. Additionally, this tests the model’s potential ability to not only denoise, but also inpaint partial observations. The experiments use the novel RePaint Heun sampler proposed in section 3.5.

4.3.1 Inferring the In-Game Score

The third experiment evaluates the model’s ability to inpaint correct in-game scores in Breakout observations. In these observations the top of the image (including the

in-game score) is masked, and the model is only presented with the playing field of the game (Figure 4.4b). The model tries to infer the correct scores based on the state of the playing field. The observations are inpainted using the proposed RePaint Heun sampler, resampling each time step $U = 3$ times.

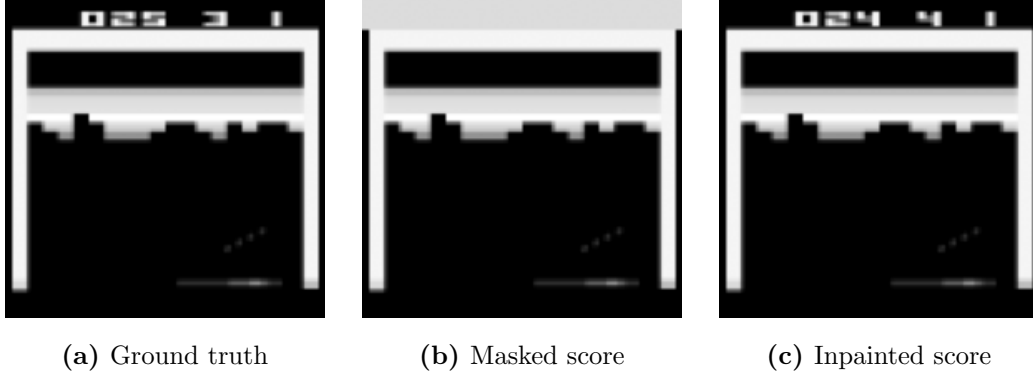
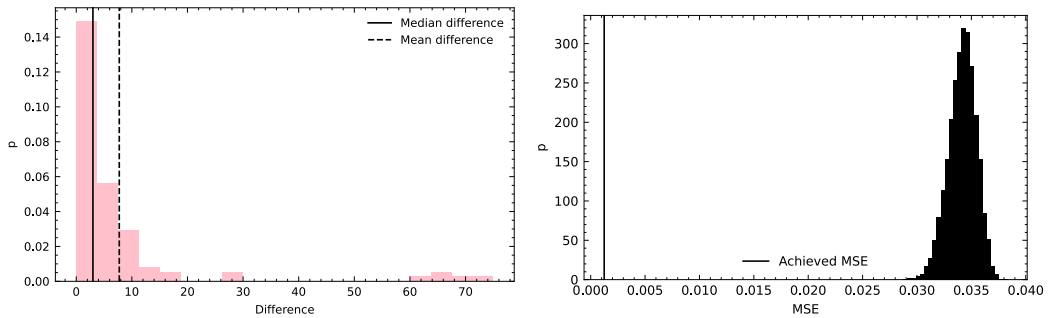


Figure 4.4: An inpainting example on an observation from Breakout. Subfigure 4.4a shows the original ground truth observation, 4.4b the observation where the top of the observation (including the area of interest) is masked and 4.4c the observation as inpainted by the trained diffusion model. This example shows only a difference of 1 point between the ground truth and predicted score.

The differences in ground truth scores and predicted scores of 100 randomly sampled observations from the test set are manually annotated, finding a mean difference of 7.7 ± 1.5 (SE) points. The original scores in the set of samples range from 0 to 397. Out of the 100 samples, 80 had a predicted difference in score below 10, of which 14 were predicted perfectly. From figure 4.5a, it can be seen that the distribution is heavily skewed towards smaller differences. The small peak between differences of 60-75 suggests a subset of frames with which the model has increased difficulties. This subset could potentially be a subset of observations from later states in the game, which are harder to achieve and thus less frequent in the memory of the agent and training dataset of the model.



(a) Distribution of difference between ground truth and predicted in-game scores in points. **(b)** Distribution of MSEs across 1000 randomly permuted frame inpainting samples.

Figure 4.5: The resulting distributions from the experiments in Section 4.2.

4.3.2 Predicting the Future

In the last experiment, the model’s ability to inpaint complete frames is evaluated by masking the last two frames of observations from Breakout. As mentioned in section 3.2, each observation consists of the four last frames from the emulator stacked on top of each other. By taking the mean of the frames, one can see the trajectory of moving objects, i.e. the ball and paddle. While the movement of the paddle is unpredictable from the observation, as it is controlled by the agent, the movement of the ball follows a predictable pattern governed by the dynamics of the environment. By letting the model inpaint the last two frames of the observation, we can inspect its ability to correctly predict future trajectories which shows awareness of environment dynamics. The number of resamples U is increased to 20, as lower values of U failed to consistently inpaint the ball, likely caused by small details having less chances to be integrated into the observation.

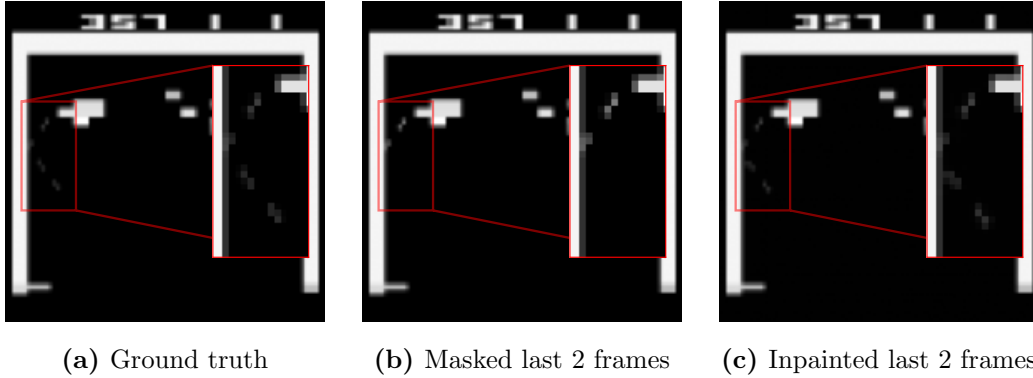


Figure 4.6: A frame inpainting example on Breakout. Subfigure 4.6a shows the original ground truth observation containing 4 stacked frames from the emulator. In subfigure 4.6b, the last two frames of the observation are masked. Subfigure 4.6c shows the last two frames inpainted using the adapted Heun RePaint sampler. The red lines present an enlargement of the area of interest (the trajectory of the ball).

Figure 4.6 shows an example where the model was able to correctly inpaint the trajectory of the ball, including the angle of the bounce on the wall. This suggests it has some degree of understanding how the ball should move under its dynamics. Only test data was used for this experiment, suggesting that the model is able to generalize well, and not just memorizing the training dataset. A larger set of examples can be found in appendix A. The mean squared error (MSE) achieved between the original frames and inpainted frames is $1.3 \cdot 10^{-3} \pm 1.2 \cdot 10^{-4}$ (SE). A permutation test is performed, permuting the images in 10000 MSE computations. Figure 4.5b shows that the found MSE is well outside the 99% highest density region of the distribution, showing that the model performs better than a random model inpainting random frames from the prior distribution.

Chapter 5

Discussion

Conclusion. In this thesis I have presented a novel framework to use diffusion models to reconstruct states from noisy and partial observations in order to improve performance in deep reinforcement learning, alongside two denoising methods for diffusion models formulated under the EDM framework [8, 9].

Results from the experiments have shown that a diffusion model trained on states of the environment is able to effectively denoise noisy observations, achieving high PSNRs (39-57 dB). The resulting diffusion agent improves achieved total reward in noisy environments by up to 82 times in Breakout and 10 times in Space Invaders for low levels of additive Gaussian noise ($\sigma_{noise} = 0.2$) compared to a vanilla DDQN agent [12, 19], and a DDQN agent using a median filter to denoise observations. This suggests that diffusion models offer an effective solution for increasing the performance of DRL agents in noisy environments by equipping them with a belief about the states of the environment. An added benefit of this framework is that it allows for a way to improve performance in scenarios where agents are presented with observations that are out of the distribution of the training states, without the need of having to re-train or finetune the underlying DQN itself.

In addition, I also presented an adaptation of Karras et al.’s Heun sampler [8] implementing the RePaint approach for diffusion-based image inpainting as introduced by Lugmayr et al. [11]. The experiments showed that the adapted sampler is able to successfully inpaint plausible details, integrating context from the whole image. Together with the trained diffusion model, it showed to be able to inpaint realistic in-game scores in masked observations of Atari Breakout, being off by only 7.7 points on average from the true scores. Furthermore, it was also able to inpaint entire Breakout frames while conserving the trajectory of moving objects in the environment. This shows that the model has gained some understanding of the underlying dynamics of the game.

To conclude, the experiments showed that for low levels of Gaussian noise, using a diffusion model can improve the performance of deep reinforcement learning agents by denoising noisy observations. Additionally, the inpainting experiments showed that the model has learned a good prior on the states of the environment by being able to inpaint in-game scores and future frames with high accuracy.

Societal Impact. The positive results of this study show the ability of diffusion models to increase robustness in DRL against noisy and partial observations. This can potentially increase the safe integration of DRL where sensor noise or partial observations might be present.

However, the proposed RePaint Heun sampler also has the potential to enhance negative biases present in the training dataset of the model, as it inpaints images using this data distribution. This has the potential to reinforce existing stereotypes and disinformation.

Machine learning and artificial intelligence are known to contribute negatively to the environment, due to the high energy cost of training models. Training the two denoising networks and deep Q-networks used in this study took a total of 208 GPU hours (not taking into account any models trained for hyperparameter search) distributed over a set of NVIDIA Quadro RTX 6000s. This consumed an estimated total of 52kWh of electricity.

Limitations. While the results show that using diffusion models can increase performance in DRL, various limitations remain. Firstly, the direct impact of the application of diffusion models was only examined in environments with observations corrupted using Gaussian noise, a task inherent to diffusion models. Secondly, the baseline median filter used in this study is a relatively weak baseline, a stronger baseline would’ve better placed the results in the frontier of the field. Furthermore, obscured and noisy observations are not the only form of partial observability. For example, in environments where state information of multiple observations needs to be integrated, the application of diffusion models is less obvious. Additionally, the use of diffusion models implies the access to a dataset of clean states, which might not always be available. And lastly, many RL applications require the need of real-time computations, outputs need to be ready while they are still relevant. Unfortunately, diffusion models often consist of very deep NNs commonly needing multiple NFE. This means that the application of diffusion might be infeasible in tasks where fast computations are necessary.

Future research. The deep collection of diffusion model literature, as well as the limitations, spark multiple interesting questions for future research. In order to address the first limitation mentioned in the previous subsection, future research could assess the direct impact of using diffusion models in environments with different types of partial observations, e.g. cropped observations, observations corrupted with different types of noise, etc. Regarding the second limitation, future research could make use of a stronger baseline, for example, a variational autoencoder [10]. Furthermore, while this study utilizes an unconditional diffusion model, a large part of image restoration literature suggests the use of conditional diffusion models. These would not model the prior distribution over states $p(s)$, but instead model a conditional distribution over states given partial observations $p(s|o)$. And lastly, future research could investigate whether using diffusion remains beneficial when the original DQN has been fine-tuned or trained on noisy observations itself.

Acknowledgements

I would like to thank dr. Luca Ambrogioni, my supervisor, for supporting me with my bachelor thesis. I would like to thank Ella Kemperman for giving me valuable technical feedback on early drafts of my thesis and providing me with useful inspiration for one of my experiments. I would like to thank dr. Renske Timmermans, my aunt, for giving me insightful feedback regarding the structure and writing of my thesis. I would like to thank Susan Jones for granting me the exception to use the high performance computing cluster. I would like to thank dr. Louis ten Bosch for taking the time to be the second reader of my bachelor thesis. Finally, I would like to give an extra special thanks to my partner, friends, and family for the constant support throughout my studies.

Bibliography

- [1] Uri M Ascher and Linda R Petzold. *Computer methods for ordinary differential equations and differential-algebraic equations*. SIAM, 1998.
- [2] Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of artificial intelligence research*, 47:253–279, 2013.
- [3] Silvio Brandi, Marco Savino Piscitelli, Marco Martellacci, and Alfonso Capozzoli. Deep reinforcement learning to optimise indoor temperature control and heating energy consumption in buildings. *Energy and Buildings*, 224:110225, 2020.
- [4] Hado Hasselt. Double q-learning. *Advances in neural information processing systems*, 23, 2010.
- [5] Matthew J Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. In *AAAI fall symposia*, volume 45, page 141, 2015.
- [6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [7] Sandy Huang, Nicolas Papernot, Ian Goodfellow, Yan Duan, and Pieter Abbeel. Adversarial attacks on neural network policies. *arXiv preprint arXiv:1702.02284*, 2017.
- [8] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. *Advances in neural information processing systems*, 35:26565–26577, 2022.
- [9] Tero Karras, Miika Aittala, Jaakko Lehtinen, Janne Hellsten, Timo Aila, and Samuli Laine. Analyzing and improving the training dynamics of diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 24174–24184, 2024.
- [10] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- [11] Andreas Lugmayr, Martin Danelljan, Andres Romero, Fisher Yu, Radu Timofte, and Luc Van Gool. Repaint: Inpainting using denoising diffusion probabilistic models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11461–11471, 2022.

- [12] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [13] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.
- [14] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015.
- [15] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. pmlr, 2015.
- [16] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019.
- [17] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.
- [18] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- [19] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.
- [20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [21] Huan Zhang, Hongge Chen, Chaowei Xiao, Bo Li, Mingyan Liu, Duane Boning, and Cho-Jui Hsieh. Robust deep reinforcement learning against adversarial perturbations on state observations. *Advances in neural information processing systems*, 33:21024–21037, 2020.
- [22] Zhengbang Zhu, Hanyu Zhao, Haoran He, Yichao Zhong, Shenyu Zhang, Haoquan Guo, Tingting Chen, and Weinan Zhang. Diffusion models for reinforcement learning: A survey. *arXiv preprint arXiv:2311.01223*, 2023.

Appendix A

Additional Results

Table A.1 presents the full list of results from the denoising experiment in section 4.1. Additionally, Table A.2 and Table A.3 showcase the full set of results of the reinforcement learning experiment presented in section 4.2. Figure A.1 and A.2 show 4 additional examples of the inpainting experiments from section 4.2.

Table A.1: The full table of results of the denoising task. Including evaluations of every noise level σ_{noise} .

G	Method	$\sigma_{noise} =$	0.1	0.2	0.3	0.4	0.5
B	Median		23.1 $\pm$ 0.2	23.0 $\pm$ 0.2	22.4 $\pm$ 0.2	21.7 $\pm$ 0.1	21.1 $\pm$ 0.1
	Diff (Naive)		59.3 $\pm$ 0.2	55.6 $\pm$ 0.2	52.2 $\pm$ 0.2	47.7 $\pm$ 0.3	44.9 $\pm$ 0.3
	Diff (Full)		59.3 $\pm$ 0.2	56.7 $\pm$ 0.3	53.7 $\pm$ 0.3	49.3 $\pm$ 0.5	45.3 $\pm$ 0.5
S	Median		26.5 $\pm$ 0.3	25.9 $\pm$ 0.2	25.2 $\pm$ 0.2	24.5 $\pm$ 0.1	23.8 $\pm$ 0.1
	Diff (Naive)		55.3 $\pm$ 0.3	50.9 $\pm$ 0.3	47.6 $\pm$ 0.3	44.7 $\pm$ 0.3	42.6 $\pm$ 0.3
	Diff (Full)		54.6 $\pm$ 0.3	51.2 $\pm$ 0.3	48.2 $\pm$ 0.4	45.0 $\pm$ 0.5	42.6 $\pm$ 0.4
$\sigma_{noise} =$			0.6	0.7	0.8	0.9	1.0
B	Median		20.6 $\pm$ 0.1	20.1 $\pm$ 0.1	19.6 $\pm$ 0.1	19.2 $\pm$ 0.1	18.7 $\pm$ 0.1
	Diff (Naive)		43.2 $\pm$ 0.2	42.0 $\pm$ 0.2	40.4 $\pm$ 0.3	39.1 $\pm$ 0.3	38.1 $\pm$ 0.3
	Diff (Full)		42.8 $\pm$ 0.4	41.3 $\pm$ 0.4	39.8 $\pm$ 0.4	38.5 $\pm$ 0.3	37.7 $\pm$ 0.5
S	Median		23.0 $\pm$ 0.1	22.3 $\pm$ 0.1	21.6 $\pm$ 0.1	20.9 $\pm$ 0.1	20.3 $\pm$ 0.1
	Diff (Naive)		41.0 $\pm$ 0.3	39.8 $\pm$ 0.2	39.1 $\pm$ 0.2	38.3 $\pm$ 0.3	37.6 $\pm$ 0.3
	Diff (Full)		40.9 $\pm$ 0.4	39.4 $\pm$ 0.3	38.4 $\pm$ 0.3	37.5 $\pm$ 0.2	36.9 $\pm$ 0.3

Table A.2: The full table of mean total achieved reward of the experiment proposed in Section 4.2.

G	Agent $\sigma_{noise} =$	0.1	0.2	0.3	0.4	0.5
B	DDQN	2.1 $\pm$ 0.3	1.1 $\pm$ 0.3	1.5 $\pm$ 0.5	1.0 $\pm$ 0.4	1.5 $\pm$ 0.4
	Median	1.6 $\pm$ 0.2	1.3 $\pm$ 0.3	1.9 $\pm$ 0.4	1.3 $\pm$ 0.4	1.3 $\pm$ 0.4
	Diff (Naive)	86.4 $\pm$ 2.8	88.7 $\pm$ 3.0	76.6 $\pm$ 4.3	20.7 $\pm$ 1.2	5.0 $\pm$ 0.7
	Diff (Full)	85.7 $\pm$ 3.7	90.3 $\pm$ 2.7	79.9 $\pm$ 4.0	31.2 $\pm$ 3.6	8.6 $\pm$ 0.7
S	DDQN	26.8 $\pm$ 1.1	12.0 $\pm$ 1.1	7.0 $\pm$ 0.9	5.3 $\pm$ 0.5	5.8 $\pm$ 0.4
	Median	18.9 $\pm$ 1.3	8.9 $\pm$ 0.9	7.8 $\pm$ 0.6	6.1 $\pm$ 0.5	6.7 $\pm$ 0.5
	Diff (Naive)	124.5 $\pm$ 8.4	122.3 $\pm$ 8.1	107.3 $\pm$ 8.9	78.4 $\pm$ 8.6	44.6 $\pm$ 3.8
	Diff (Full)	117.6 $\pm$ 7.9	121.4 $\pm$ 7.7	102.5 $\pm$ 9.1	79.0 $\pm$ 7.3	54.0 $\pm$ 5.3
	$\sigma_{noise} =$	0.6	0.7	0.8	0.9	1.0
B	DDQN	1.3 $\pm$ 0.3	1.0 $\pm$ 0.3	1.1 $\pm$ 0.4	0.8 $\pm$ 0.3	1.6 $\pm$ 0.4
	Median	1.9 $\pm$ 0.5	1.7 $\pm$ 0.4	1.9 $\pm$ 0.5	1.2 $\pm$ 0.4	1.3 $\pm$ 0.4
	Diff (Naive)	2.3 $\pm$ 0.3	1.1 $\pm$ 0.2	1.3 $\pm$ 0.1	1.2 $\pm$ 0.1	1.5 $\pm$ 0.2
	Diff (Full)	3.9 $\pm$ 0.4	2.4 $\pm$ 0.3	2.3 $\pm$ 0.4	1.3 $\pm$ 0.2	1.4 $\pm$ 0.2
S	DDQN	5.5 $\pm$ 0.5	5.4 $\pm$ 0.5	5.7 $\pm$ 0.4	6.3 $\pm$ 0.6	6.4 $\pm$ 0.6
	Median	6.5 $\pm$ 0.5	6.6 $\pm$ 0.4	6.3 $\pm$ 0.5	7.2 $\pm$ 0.4	7.6 $\pm$ 0.4
	Diff (Naive)	38.3 $\pm$ 3.6	36.7 $\pm$ 3.2	31.1 $\pm$ 2.1	25.8 $\pm$ 2.1	20.5 $\pm$ 1.7
	Diff (Full)	39.6 $\pm$ 3.1	37.7 $\pm$ 4.7	31.4 $\pm$ 2.3	30.0 $\pm$ 2.8	27.2 $\pm$ 1.7

Table A.3: The full table of mean episode length of the experiment proposed in Section 4.2.

G	Agent	$\sigma_{noise} =$	0.1	0.2	0.3	0.4	0.5
B	DQNN		227.6 $\pm$ 13.3	194.0 $\pm$ 10.7	209.6 $\pm$ 17.7	192.9 $\pm$ 15.5	206.8 $\pm$ 14.4
	Median		220.3 $\pm$ 9.5	220.3 $\pm$ 10.0	234.2 $\pm$ 15.6	219.4 $\pm$ 13.4	211.6 $\pm$ 14.1
	Diff (Naive)		1910.2 $\pm$ 69.5	1997.8 $\pm$ 94.1	1857.7 $\pm$ 90.7	795.0 $\pm$ 35.0	304.0 $\pm$ 23.4
	Diff (Full)		1914.5 $\pm$ 80.6	2118.4 $\pm$ 89.2	1757.4 $\pm$ 71.5	913.9 $\pm$ 36.3	423.9 $\pm$ 22.9
S	DQNN		824.1 $\pm$ 38.1	657.5 $\pm$ 34.7	562.0 $\pm$ 31.8	536.1 $\pm$ 31.4	536.9 $\pm$ 28.5
	Median		592.9 $\pm$ 38.0	469.6 $\pm$ 30.7	523.9 $\pm$ 30.4	505.4 $\pm$ 17.4	532.8 $\pm$ 23.8
	Diff (Naive)		2288.6 $\pm$ 148.0	2308.1 $\pm$ 136.0	2059.4 $\pm$ 156.2	1548.4 $\pm$ 156.3	936.0 $\pm$ 72.6
	Diff (Full)		2211.4 $\pm$ 131.3	2267.9 $\pm$ 117.5	2012.1 $\pm$ 160.2	1547.5 $\pm$ 125.3	1113.8 $\pm$ 94.9
$\sigma_{noise} =$			0.6	0.7	0.8	0.9	1.0
B	DQNN		203.2 $\pm$ 12.0	190.7 $\pm$ 11.7	203.2 $\pm$ 14.5	190.7 $\pm$ 10.2	211.7 $\pm$ 12.9
	Median		238.2 $\pm$ 16.4	227.0 $\pm$ 14.9	237.0 $\pm$ 17.4	217.8 $\pm$ 14.0	215.7 $\pm$ 13.0
	Diff (Naive)		197.3 $\pm$ 9.8	160.2 $\pm$ 6.3	161.1 $\pm$ 4.0	157.2 $\pm$ 4.1	169.8 $\pm$ 6.4
	Diff (Full)		260.4 $\pm$ 13.4	209.0 $\pm$ 12.0	213.0 $\pm$ 15.8	173.4 $\pm$ 8.0	174.9 $\pm$ 9.1
S	DQNN		499.4 $\pm$ 19.0	519.5 $\pm$ 15.8	500.7 $\pm$ 25.3	511.5 $\pm$ 27.9	552.3 $\pm$ 38.3
	Median		535.0 $\pm$ 23.7	565.8 $\pm$ 28.7	528.0 $\pm$ 19.2	543.4 $\pm$ 10.9	576.4 $\pm$ 25.1
	Diff (Naive)		813.0 $\pm$ 68.1	791.6 $\pm$ 63.1	716.8 $\pm$ 44.3	626.5 $\pm$ 40.6	605.0 $\pm$ 39.9
	Diff (Full)		864.7 $\pm$ 58.2	813.2 $\pm$ 85.1	694.2 $\pm$ 48.7	701.6 $\pm$ 57.5	628.9 $\pm$ 38.8

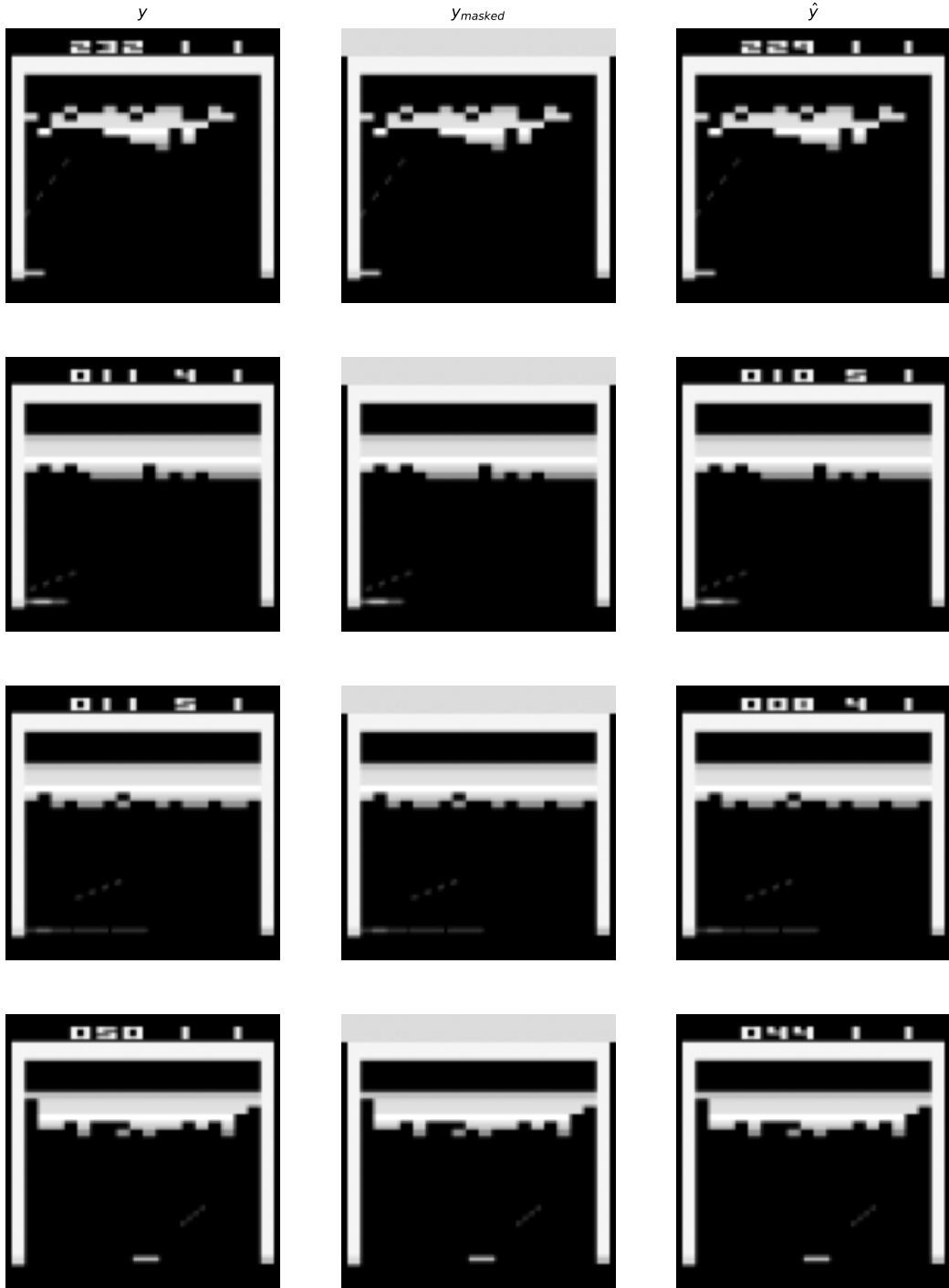


Figure A.1: Example observations of the in-game score inpainting experiment in Section 4.3.1.

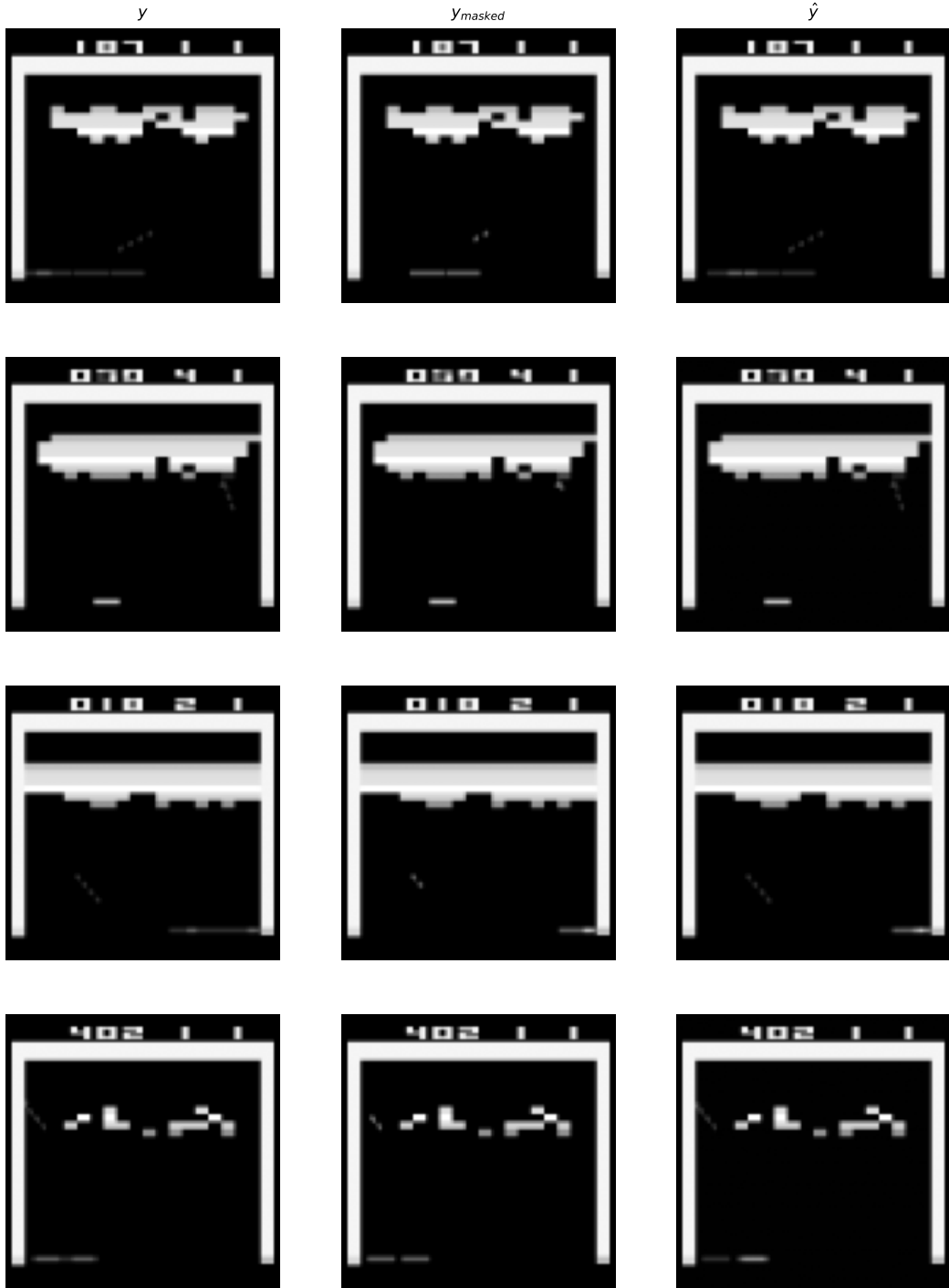


Figure A.2: Example observations of the frames inpainting experiment in Section 4.3.2

Appendix B

Deep Reinforcement Learning

B.1 Preprocessing

This thesis follows the exact preprocessing steps as proposed by Mnih et al. [12]. Firstly, the RGB channels of the input frame from the emulator are transformed into a single greyscale channel, and the size of the frame is downscaled to 84×84 . The last four processed frames from the emulator are stacked on top of each other and presented to the agent as the state of the environment. Actions are only performed on every 4th frame, and repeated on the skipped frames. The 4 stacked frames and repeated actions are referred to as a single environment step.

B.2 Training

The DDQN agents learn by optimizing the loss function specified in Equation B.1, which was introduced by Hasselt et al. [19].

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')}[(r + \gamma Q_{\theta'}(s', \arg \max_{a'} Q_{\theta}(s', a')) - Q_{\theta}(s, a))^2] \quad (\text{B.1})$$

In this equation, (s, a, r, s') is a batch of experience tuples sampled from the agents replay memory, γ the discount factor, θ the weights of the DQN, and θ' the weights of the target network. Algorithm 2 presents the complete training algorithm, adapted from Mnih et al. [12].

B.3 Network Architecture

The network architecture used in this thesis is the exact convolutional neural network based architecture proposed by Mnih et al. [12].

B.4 Hyperparameters

The hyperparameters used for training and application of the DDQN agents can be found in Table B.1.

Algorithm 2 Deep Q-learning with Experience Replay and Double Q-learning

- 1: Initialize replay memory $\mathcal{D}$ to capacity N
 - 2: Initialize action-value function Q_θ with random weights θ
 - 3: **for** episode = 1, M **do**
 - 4: **for** $t = 1, T$ **do**
 - 5: With probability ϵ select a random action a_t
 - 6: otherwise select $a_t = \arg \max_a Q_\theta(s_t, a)$
 - 7: Execute action a_t in the emulator and observe reward r_t
 - 8: Set $s_{t+1} = s_t$
 - 9: Store transition (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$
 - 10: Sample random minibatch of transitions (s_j, a_j, r_j, s_{j+1})
 - 11: Set $y_j = \begin{cases} r_j & \text{for terminal } s_{j+1} \\ r_j + \gamma Q_{\theta'}(s_j, \arg \max_a Q_\theta(s_j, a)) & \text{for non-terminal } \phi_{j+1} \end{cases}$
 - 12: Perform a gradient descent step on $\|y_j - Q_\theta(s_j, a_j)\|_2^2 \quad \triangleright \text{Eq. B.1}$
-

Table B.1: Table adapted from Mnih et al. [13]

Hyperparameter	Value	Description
minibatch size	32	Number of experience samples used during a single gradient update.
replay memory size	1000000	The experience is sampled from this number of most recent states.
agent history length	4	The number of stacked frames in a single state.
target network update frequency	10000	The frequency of environment steps at which the weights of the DQN are copied to the target network.
discount factor	0.99	Discount factor γ used in Q-learning updates
action repeat	4	Repeat each action selected by the agent this many times. Equal to the agent history length, and equal 1 environment step.
update frequency	4	The number of environment steps between successive gradient updates.
learning rate	$2.5 \cdot 10^{-5}$	The learning rate used by AdamW.
initial exploration	1	Initial value of ϵ in ϵ -greedy exploration.
final exploration	0.1	Final value of ϵ in ϵ -greedy exploration.
final exploration frame	1000000	The number of frames over which the initial value of ϵ is linearly annealed to its final value.
replay start size	50000	A uniform random policy is run for this number of frames before learning starts and the resulting experience is used to populate the replay memory.
no-op max	30	Maximum number of "do nothing" actions performed by the agent at the start of an episode.

Appendix C

Diffusion Models

C.1 Sampling

Karras et al. [8] showed that the score is related to the denoising network according to Equation C.1.

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}; \sigma) = (D_{\theta}(\mathbf{x}; \sigma) - \mathbf{x})/\sigma^2 \quad (\text{C.1})$$

Algorithm 3 presents the sampler proposed by Karras et al. [8], using Heun’s 2nd order method and the denoising network $D_{\theta}(\mathbf{x}; \sigma)$.

Algorithm 3 Deterministic sampling using Heun’s 2nd order method with arbitrary $\sigma(t)$ and $s(t)$.

```

1: procedure HEUN_SAMPLER( $D_{\theta}(\mathbf{x}; \sigma), \sigma(t), s(t), t_{i \in \{0, \dots, N\}}\rangle$ )
2:    $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \sigma^2(t_0)s^2(t_0)\mathbf{I})$ 
3:   for  $i \in 0, N - 1$  do
4:      $\mathbf{d}_i \leftarrow (\frac{\dot{\sigma}(t_i)}{\sigma(t_i)} + \frac{\dot{s}(t_i)}{s(t_i)})\mathbf{x}_i - \frac{\dot{\sigma}(t_i)s(t_i)}{\sigma(t_i)}D_{\theta}(\frac{\mathbf{x}_i}{s(t_i)}; \sigma(t_i))$ 
5:      $\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + (t_{i+1} - t_i)\mathbf{d}_i$ 
6:     if  $\sigma(t_{i+1}) \neq 0$  then
7:        $\mathbf{d}_i \leftarrow (\frac{\dot{\sigma}(t_{i+1})}{\sigma(t_{i+1})} + \frac{\dot{s}(t_{i+1})}{s(t_{i+1})})\mathbf{x}_{i+1} - \frac{\dot{\sigma}(t_{i+1})s(t_{i+1})}{\sigma(t_{i+1})}D_{\theta}(\frac{\mathbf{x}_{i+1}}{s(t_{i+1})}; \sigma(t_{i+1}))$ 
8:        $\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + (t_{i+1} - t_i)(\frac{1}{2}\mathbf{d}_i + \frac{1}{2}\mathbf{d}_i')$ 
   return  $\mathbf{x}_N$ 
```

C.2 Training

During training, the denoising network $D_{\theta}(\mathbf{x}, \sigma)$ is trained to remove randomly sampled noise n added to samples y from the dataset. It learns by minimizing the weighted L2 loss described in Equation C.2b.

$$n \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}) \quad (\text{C.2a})$$

$$\mathcal{L}(\theta) = \mathbb{E}_{\sigma, \mathbf{y}, n} [\lambda(\sigma) \|D_{\theta}(\mathbf{y} + \mathbf{n}, \sigma) - \mathbf{y}\|_2^2] \quad (\text{C.2b})$$

The function $\lambda(\sigma)$ in Equation C.2b weights the loss of samples in relation to the level of added noise σ .

C.3 Network and Training Configuration

Table C.1: Table adapted from Karras et al. [8].

Network & Training Configuration	
Sampling	
ODE solver	2 nd order method
Timesteps (Discretisation)	$(\sigma_{max}^{\frac{1}{\rho}} + \frac{i}{N-1}(\sigma_{min}^{\frac{1}{\rho}} - \sigma_{max}^{\frac{1}{\rho}}))^{\rho}$
Noise schedule $\sigma(t)$	t
Scaling $s(t)$	1
Preconditioning	
Skip scaling $c_{skip}(\sigma)$	$\sigma_{data}^2 / (\sigma^2 + \sigma_{data}^2)$
Output scaling $c_{out}(\sigma)$	$\sigma \cdot \sigma_{data} / \sqrt{\sigma_{data}^2 + \sigma^2}$
Input scaling $c_{in}(\sigma)$	$1 / \sqrt{\sigma_{data}^2 + \sigma^2}$
Noise cond. $c_{noise}(\sigma)$	$\frac{1}{4} \ln(\sigma)$
Network	
Input resolution	88×88
Input channels	4
Channel multiplier	128
Channels per resolution	2-2-2
Attention resolutions	22
Number of residual blocks per resolution	3
Dropout probability	0%
Training	
Noise distribution	$\ln(\sigma) \sim \mathcal{N}(P_{mean}, P_{std}^2)$
Loss weighting $\lambda(\sigma)$	$(\sigma^2 + \sigma_{data}^2) / (\sigma \cdot \sigma_{data})^2$
Learning rate	10^{-5}
Minibatch size	32
Parameters	
$\sigma_{min} = 0.002, \sigma_{max} = 80, \sigma_{data} = 0.5, \rho = 7, P_{mean} = -1.2, P_{std} = 1.2$	

Appendix D

Denoising Methods

D.1 Full Denoising

D.1.1 Algorithm

Algorithm 4 Denoising using the HeunSampler with $x_0 = o_{noisy}$ and $\sigma_{max} = \sigma_{noise}$

```

1: procedure FULLDENOISING( $\mathbf{o}_{noisy}, \sigma_{noise}, D_\theta(\mathbf{x}; \sigma), \sigma(t), s(t), t_{i \in \{0, \dots, N\}}$ )
2:    $\mathbf{x}_0 \leftarrow \mathbf{o}_{noisy}$ 
3:   for  $i \in 0, N - 1$  do
4:      $\mathbf{d}_i \leftarrow (\frac{\dot{\sigma}(t_i)}{\sigma(t_i)} + \frac{\dot{s}(t_i)}{s(t_i)})\mathbf{x}_i - \frac{\dot{\sigma}(t_i)s(t_i)}{\sigma(t_i)}D_\theta(\frac{\mathbf{x}_i}{s(t_i)}; \sigma(t_i))$ 
5:      $\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + (t_{i+1} - t_i)\mathbf{d}_i$ 
6:     if  $\sigma(t_{i+1}) \neq 0$  then
7:        $\mathbf{d}_i \leftarrow (\frac{\dot{\sigma}(t_{i+1})}{\sigma(t_{i+1})} + \frac{\dot{s}(t_{i+1})}{s(t_{i+1})})\mathbf{x}_{i+1} - \frac{\dot{\sigma}(t_{i+1})s(t_{i+1})}{\sigma(t_{i+1})}D_\theta(\frac{\mathbf{x}_{i+1}}{s(t_{i+1})}; \sigma(t_{i+1}))$ 
8:      $\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + (t_{i+1} - t_i)(\frac{1}{2}\mathbf{d}_i + \frac{1}{2}\mathbf{d}_i')$ 
  return  $\mathbf{x}_N$ 

```

D.1.2 Hyperparameter Search

The full denoising method (Section 3.4) alters the Heun sampler by starting from the noisy observation o and setting σ_{max} to the level of noise in the observation $\sigma_{max} = \sigma_{noise}$, instead of sampling pure Gaussian noise with $\sigma_{max} = 80$ (see Table C.1 for the full configuration). As the noise levels in the experiment are substantially lower than the proposed starting noise level $\sigma_{noise} \ll 80$, it is likely that the number of required sampler steps for acceptable results also lies lower. From the elbow plot in Figure D.1 it can be seen that the PSNR starts to plateau at $N \geq 4$ and fully plateaus for all tested noise levels around $N \geq 8$. To make sure no small details are lost during denoising, N is therefore set to 8 during the experiments. However, one could set $N = 4$, trading of denoising quality for efficiency.

D.2 Naive Denoising

The naive denoising method directly makes use of the denoising network by conditioning it on the noise present in the observation σ_{noise} . The method for de-

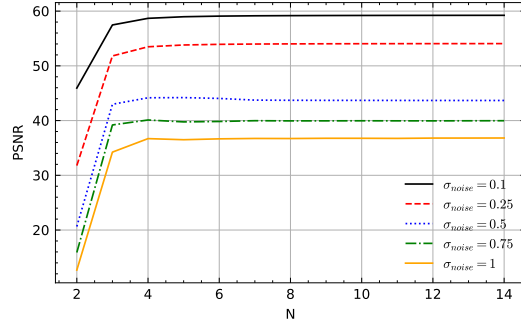


Figure D.1: The elbow plot displaying the peak signal-to-noise ratio in decibel (dB) as a function of sampler steps N .

noising a noisy observation $\mathbf{o}_{noisy}$ with noise level σ_{noise} , therefore simply becomes:
 $f(\mathbf{o}_{noisy}, \sigma_{noise}) = D_{\theta}(\mathbf{o}_{noisy}, \sigma_{noise})$.